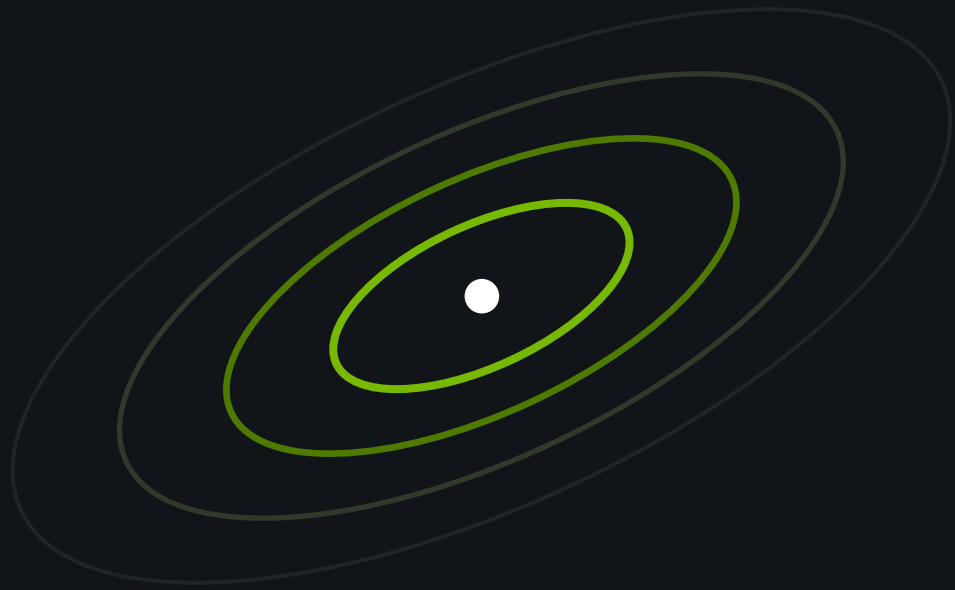
RL FOR REAL · DEEP RL II

Don't Move Too Far

Trust regions, PPO, and policy gradients today



Gal Dalal · NVIDIA Research · Oct 11, 2026

RECAP

Where we left off

$$\nabla_\theta J(\theta) = \mathbb{E}\left[\nabla_\theta \log \pi_\theta(a|s)\ \hat{A}(s,a)\right]$$

∇ log π which way to push each action

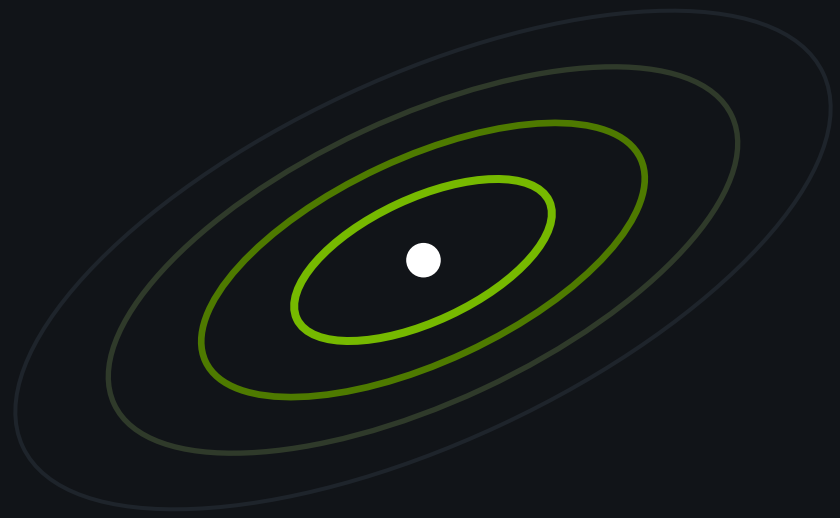
Â how much — critic + GAE (Ido)

Today: how big a step can we safely take?

01

Why step size is everything

In RL, one bad update ruins the next batch of data



In RL, your data comes from your policy

Supervised learning

Fixed dataset.

A bad step costs one step.

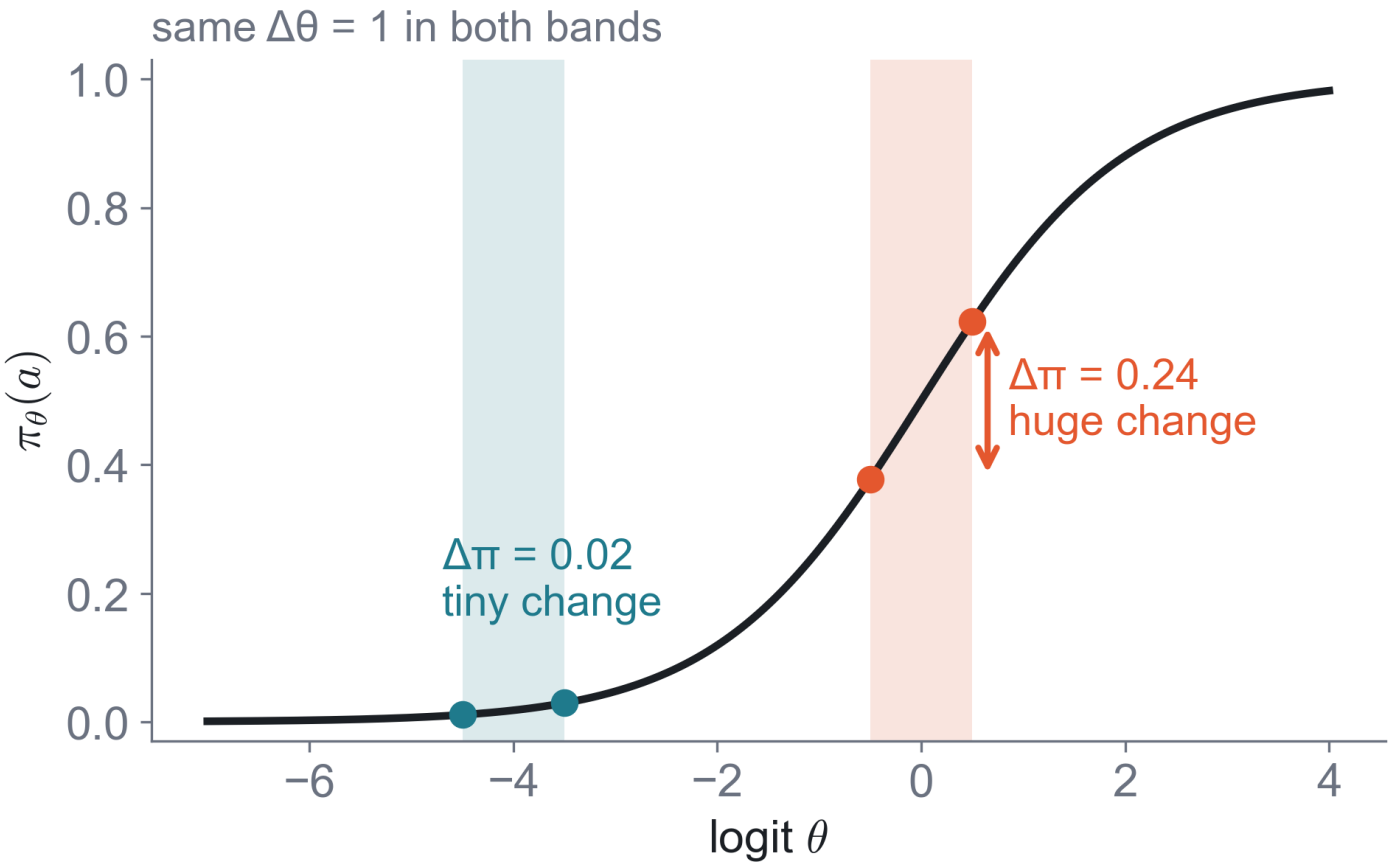
Reinforcement learning

π → data → gradient → π

A bad step poisons the next batch.



A small step in θ can be a huge step in π



Just lower the LR?

- Too slow where π is flat
- Still unsafe where π is steep

Measure the step in **policy space**, not parameter space

Measure steps by KL, not by ‖Δθ‖

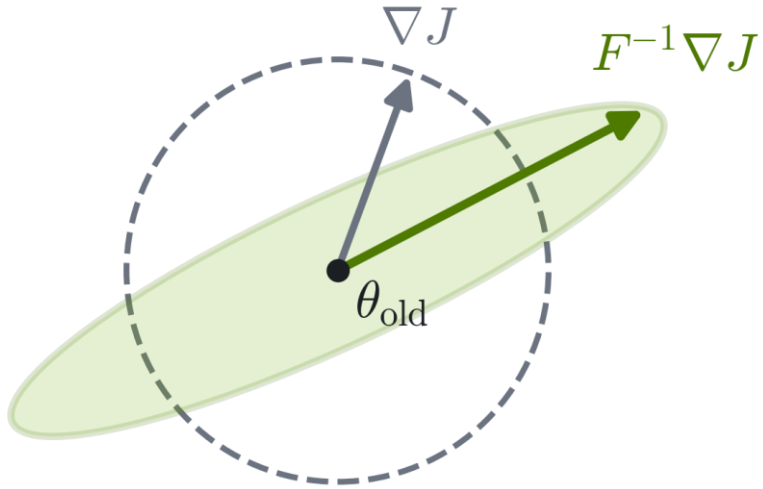
$$\max_{\Delta\theta} \nabla J^\top \Delta\theta \quad \text{s.t.} \quad \bar{D}_{\mathrm{KL}}(\pi_\theta \,\|\, \pi_{\theta+\Delta\theta}) \le \delta$$

$$\Delta\theta \propto F^{-1}\nabla_\theta J, \qquad F = \mathbb{E}_{s,\, a\sim\pi_\theta}[\nabla \log \pi_\theta \nabla \log \pi_\theta^\top]$$

mean KL = KL averaged over visited states

F = Fisher information = Hessian of the mean KL

→ the natural gradient (Kakade, 2001)



The surrogate L(θ): score a new policy on old data

$$L(\theta) = \mathbb{E}_{s \sim d^{\pi_{\text{old}}},\ a \sim \pi_{\text{old}}(\cdot|s)} \left[r_\theta(s,a)\, \hat{A}(s,a) \right], \qquad r_\theta(s,a) = \frac{\pi_\theta(a|s)}{\pi_{\text{old}}(a|s)}$$

Old-policy data

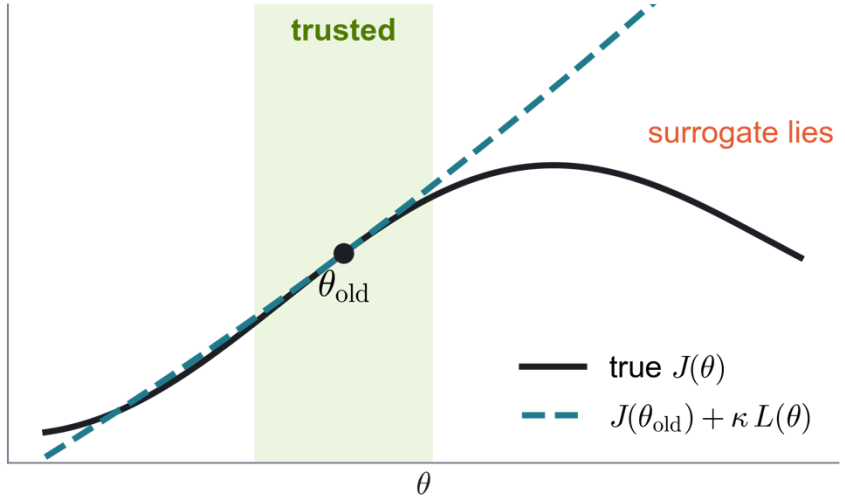
reuse states & actions sampled by π_old

Probability ratio r_θ

how much likelier a becomes under π_θ

Advantage Â

was a better or worse than expected?



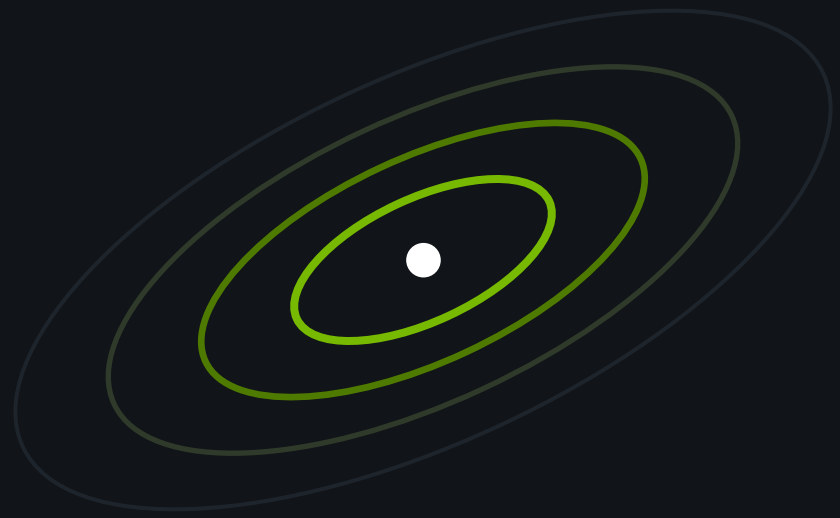
- **L(θ_old) = 0** (exact A): L measures improvement, not return
- **First order:** J(θ) ≈ J(θ_old) + κ·L(θ), κ = 1/(1−γ)
- **Far from θ_old:** values diverge, trust only nearby

d^π_old: normalized discounted state occupancy of π_old. With estimated Â, these identities hold only approximately.

02

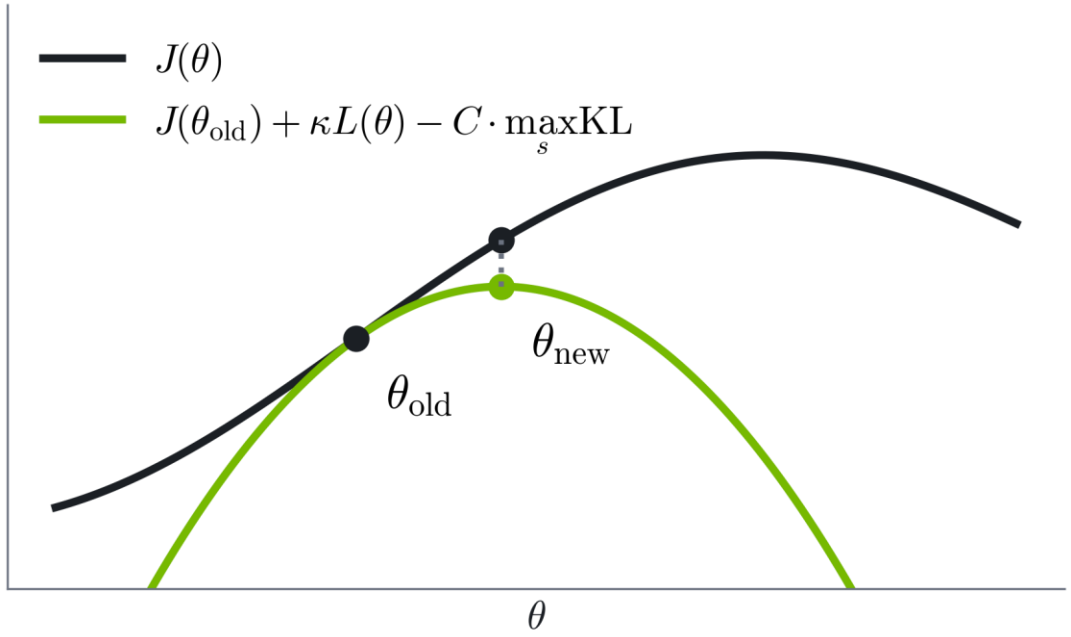
TRPO

Turn “stay close” into a constraint



A guarantee: maximize a lower bound

$$J(\pi_\theta) \geq J(\pi_{\text{old}}) + \kappa L(\theta) - C \cdot \max_s D_{\text{KL}}\left(\pi_{\text{old}}(\cdot|s) \,\|\, \pi_\theta(\cdot|s)\right)$$



- Tight at θ_old (L = 0, KL = 0)
- Raise the bound → J can only go up
- **Monotonic improvement, in theory**

Needs exact A and max-state KL. Practical TRPO (estimated Â, mean KL) has no exact guarantee.

Kakade & Langford, 2002; Schulman et al., 2015 (Thm. 1)

TRPO: a KL-constrained step

$$\max_{\theta} L(\theta) \quad \text{s. t.} \quad \bar{D}_{\mathrm{KL}}\left(\pi_{\mathrm{old}} \,\|\, \pi_{\theta}\right) \leq \delta$$

max KL → **mean KL**

estimable from samples

penalty C → **constraint δ**

δ is in KL units: one value (≈ 0.01)

works across tasks

Solving it in practice

1

Rollout

collect data with π_old, estimate the gradient

$$g = \nabla_\theta L(\theta_{\text{old}})$$

2 ~10 iterations

Conjugate gradient

iteratively solve F x = g
each iteration: one F·v by autodiff; F is never formed

$$x \approx F^{-1} g$$

3

Scale

make the quadratic KL model exactly δ

$$\Delta\theta = \sqrt{\frac{2\delta}{x^\top F x}}\, x$$

4

Line search

check actual mean KL ≤ δ and L improves; else shrink

Δθ, ½Δθ, ¼Δθ, …

CG is an inner solver, not training: the policy changes once per update, after the line search.

Schulman et al., 2015

Why the field moved on

Expensive

CG + line search
on every update

Rigid

awkward with shared
actor-critic nets, dropout

Off the SGD path

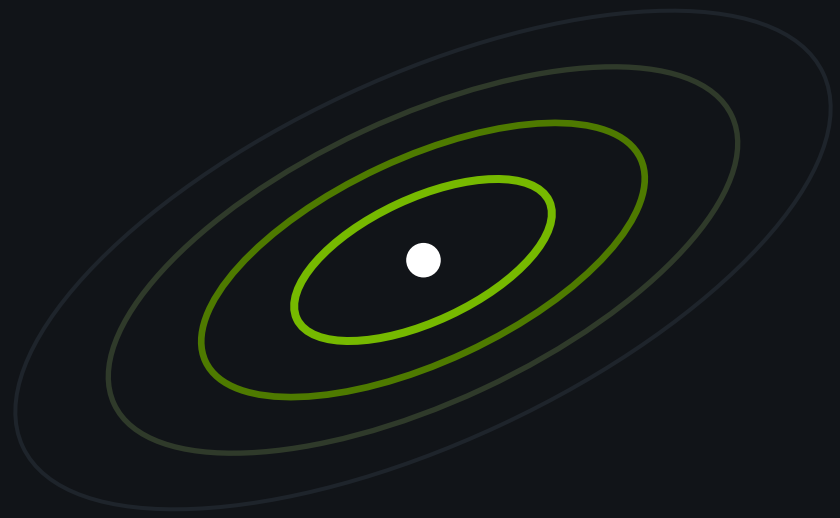
doesn't fit minibatch
Adam training

Can we get TRPO's stability with plain first-order SGD? → PPO

03

PPO

TRPO's idea at first-order prices



Option A: back to a penalty, with β tuned to hit δ

$$\max_{\theta} L(\theta) - \beta \bar{D}_{\mathrm{KL}}\left(\pi_{\mathrm{old}} \,\|\, \pi_{\theta}\right)$$

Adaptive β

KL > 1.5 δ	→	**β × 2**
KL < δ / 1.5	→	**β ÷ 2**

β = the constraint's multiplier, found by trial instead of CG.

Same target δ, first-order only.

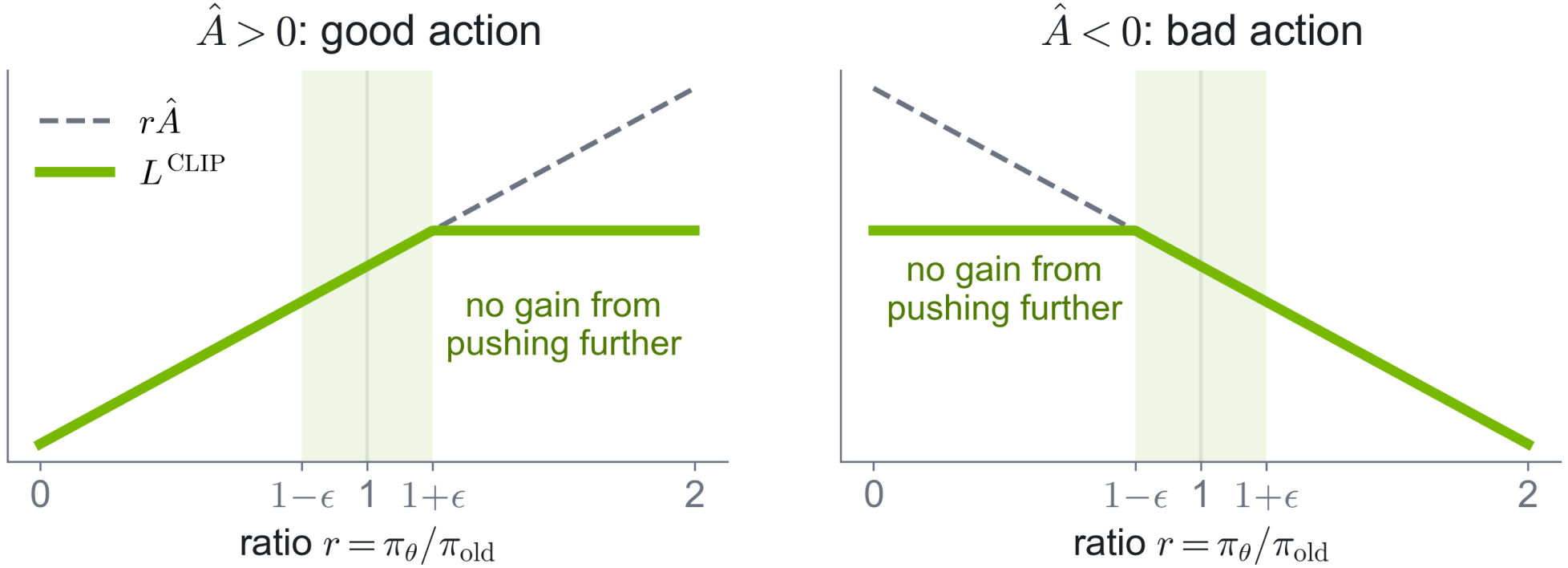
Keep this form in mind

A similar KL-regularized objective returns in RLHF, with KL to a frozen **π_ref**, not π_old.

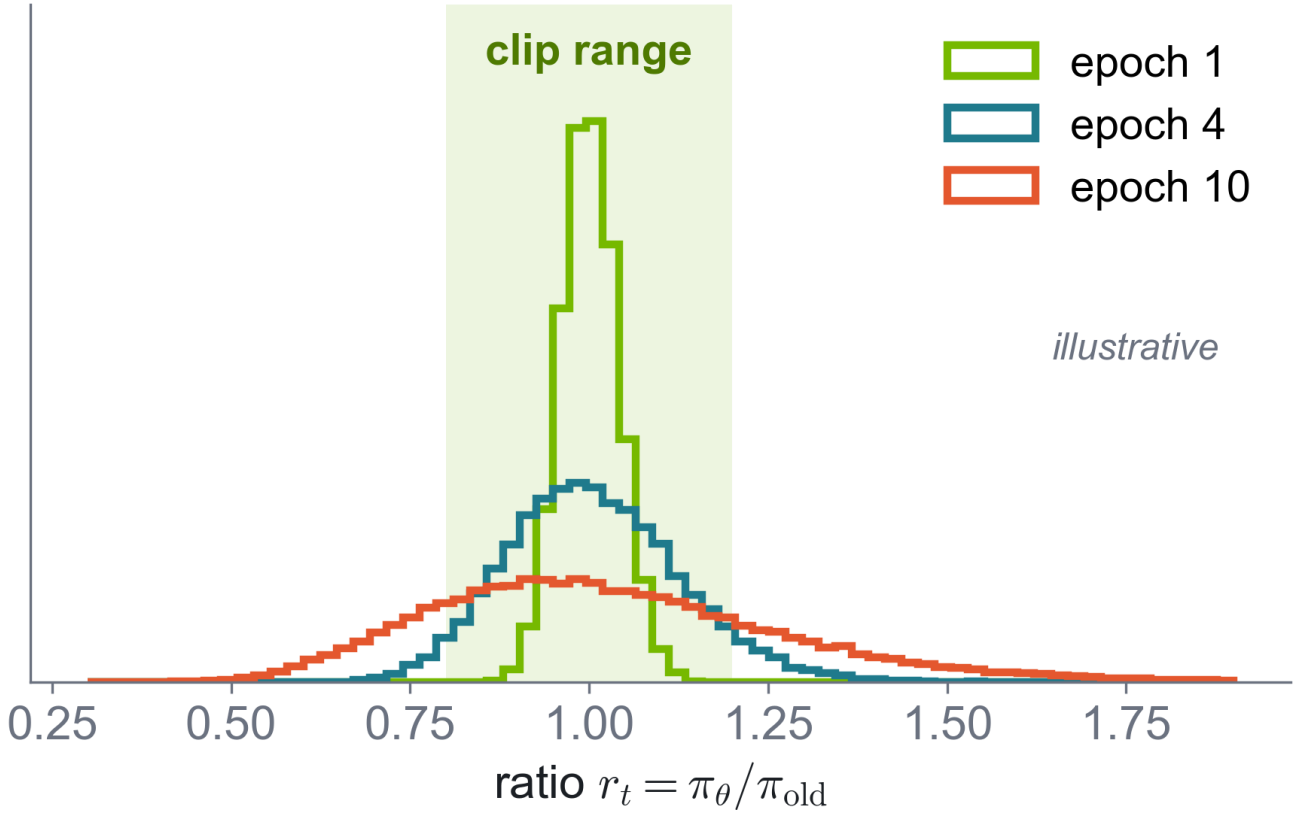
DPO is derived from that RLHF objective.

Option B: clip the ratio

$$L^{\text{CLIP}}(\theta) = \mathbb{E}\left[\min\left(r_t \hat{A}_t,\ \text{clip}(r_t,\, 1-\epsilon,\, 1+\epsilon)\,\hat{A}_t\right)\right]$$



Clipping is not a true trust region



- Outside the clip: zero gradient
- …but nothing pulls r back
- Over K epochs, ratios and KL drift
- **Fix: watch KL, stop early**

Wang et al., 2019; Engstrom et al., 2020

The whole algorithm

```
for it in range(100):
    rollout = collect(policy, N=4096)
    A, R = gae(rollout, γ, λ)
    A = (A - A.mean()) / A.std()
    for epoch in range(10):   # reuse!
        for mb in minibatches(256):
            loss = ppo_loss(old_logp, logp, A)
                 + 0.5 * value_loss(V, R)
            loss.backward()
            clip_grad_norm(0.5); step()
```

vs. REINFORCE

- + critic V(s) and GAE
- + 10 epochs on the same data
- → **needs the ratio and the clip**

Entropy bonus: common in libraries, not in toy-ppo

Implementation details matter

37

implementation details of PPO

Huang et al., 2022

Much of PPO's edge over TRPO came from **code-level choices**, not the clip

Engstrom et al., 2020

- Advantage normalization
- Obs / reward normalization
- Orthogonal init
- LR annealing
- Grad-norm clipping
- Timeout ≠ terminal
- Value clipping (often hurts)
- Separate actor & critic

PPO

What to watch while it trains

Rules of thumb — you'll read these curves all week

approx KL

≈ 0.01 – 0.02



clip fraction

≈ 0.1 – 0.3



explained variance

→ 1



entropy

slow, steady decay



value loss

goes down



episode return

what you actually want



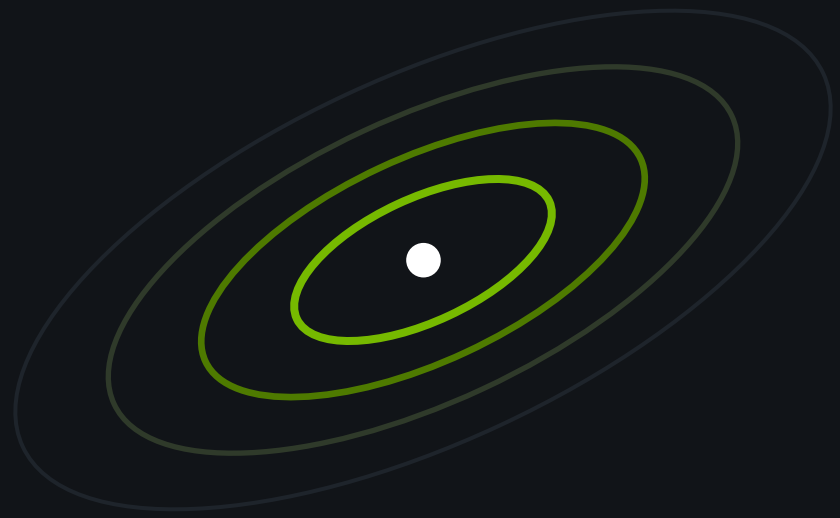
Every knob sets the step size

Knob	toy-ppo	Turn it up →
clip ε	`0.2`	bigger steps, less stable
epochs K	`10`	more data reuse, more drift
learning rate	`1e-4`	bigger steps
minibatch	`256`	less noise, fewer updates
rollout N	`4096`	better estimates, slower iterations
GAE λ	`1.0`	less bias, more variance

04

Policy gradients today

Same ideas, new scales: simulation, off-policy, LLMs



PG TODAY

PPO at scale: massively parallel simulation

1000s
parallel envs

1
GPU

minutes
to learn to walk

Rudin et al., 2021 — Isaac Gym

- Samples nearly free → on-policy waste doesn't matter
- Huge batches, short rollouts, very stable

Up next in the week

RL for Robotics

Chen Tessler & Shir Gur
Thursday

PG TODAY

Off-policy policy gradients: SAC

	PPO	SAC
Data	on-policy rollouts	replay buffer
Objective	clipped surrogate	reward + entropy
Gradient	likelihood ratio	reparameterized via Q
Best when	samples are cheap	samples are expensive
Watch out	sample hungry	tuning-sensitive

Haarnoja et al., 2018 (SAC); Fujimoto et al., 2018 (TD3)

PG TODAY

PPO for LLMs

state	prompt + tokens so far
action	next token
reward	RM / verifier, at the end

$$\max_{\theta} \mathbb{E}_{y \sim \pi_\theta(\cdot|x)}[r(x,y)] \; - \; \beta \mathrm{KL}(\pi_\theta \,\|\, \pi_{\mathrm{ref}})$$

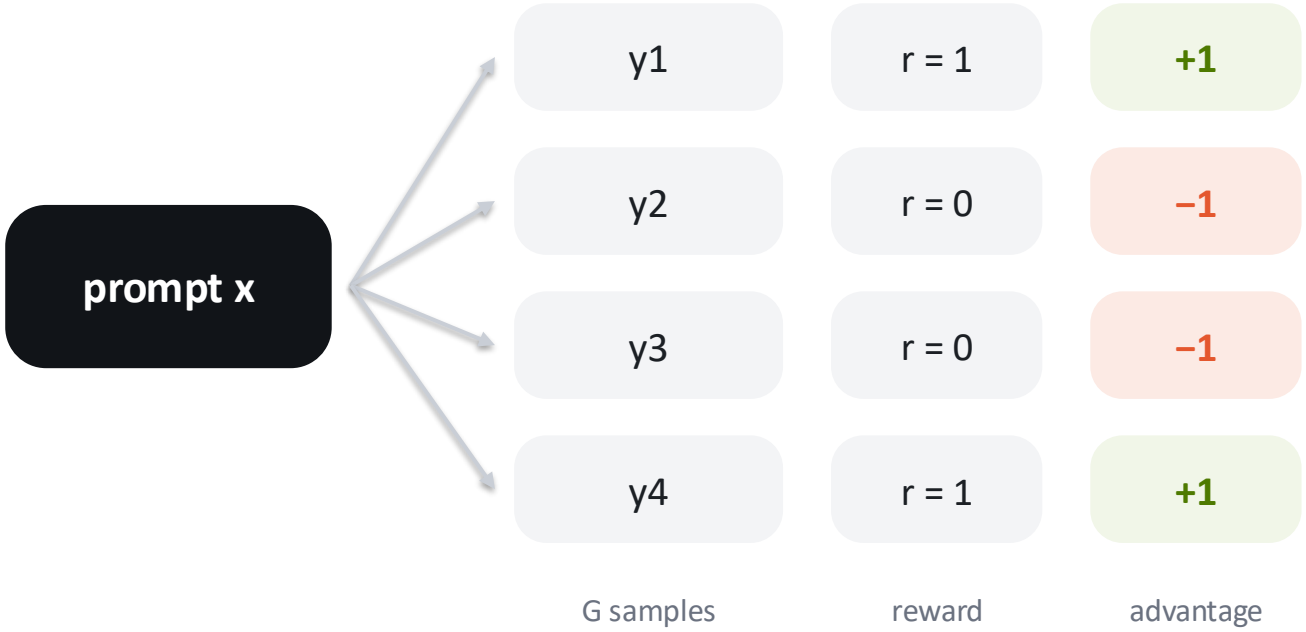
KL to π_ref ≠ KL to π_old
stay sane vs. step small

Four models in memory

Policy | **Reference** | **Reward model** | **Critic**

the pain point

GRPO: replace the critic with a group



$$\hat{A}_i = \frac{r_i - \text{mean}(r_1, \ldots, r_G)}{\text{std}(r_1, \ldots, r_G)}$$

Everything else is PPO

- ratio · clip · KL to π_ref
- no critic, no value loss

Group mean = Monte-Carlo baseline
(Ido's lecture). RLOO: leave-one-out mean

Shao et al., 2024 (DeepSeekMath); Ahmadian et al., 2024 (RLOO)

PG TODAY

GRPO in practice

Why it took over

- Verifiable rewards: math, code
- No critic → less memory, less tuning
- Many samples per prompt are cheap

Known fixes

Dr. GRPO
drop std & length normalization biases

DAPO
clip-higher, skip all-same-reward groups

Liu et al., 2025 (Dr. GRPO); Yu et al., 2025 (DAPO)

You'll compare PPO vs. GRPO in the LLM exercise

DPO: skip the RL loop

1 optimal RLHF policy

$$\pi^*(y|x) \propto \pi_{\text{ref}}(y|x)\exp\left(r(x,y)/\beta\right)$$

2 solve for the reward

$$r(x,y) = \beta\log\frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \beta\log Z(x)$$

3 Bradley–Terry (Z cancels)

$$P(y_w \succ y_l) = \sigma(r(x,y_w) - r(x,y_l))$$

$$\mathcal{L}_{\text{DPO}} = -\log\sigma\left(\beta\log\frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta\log\frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right)$$

Trade-off

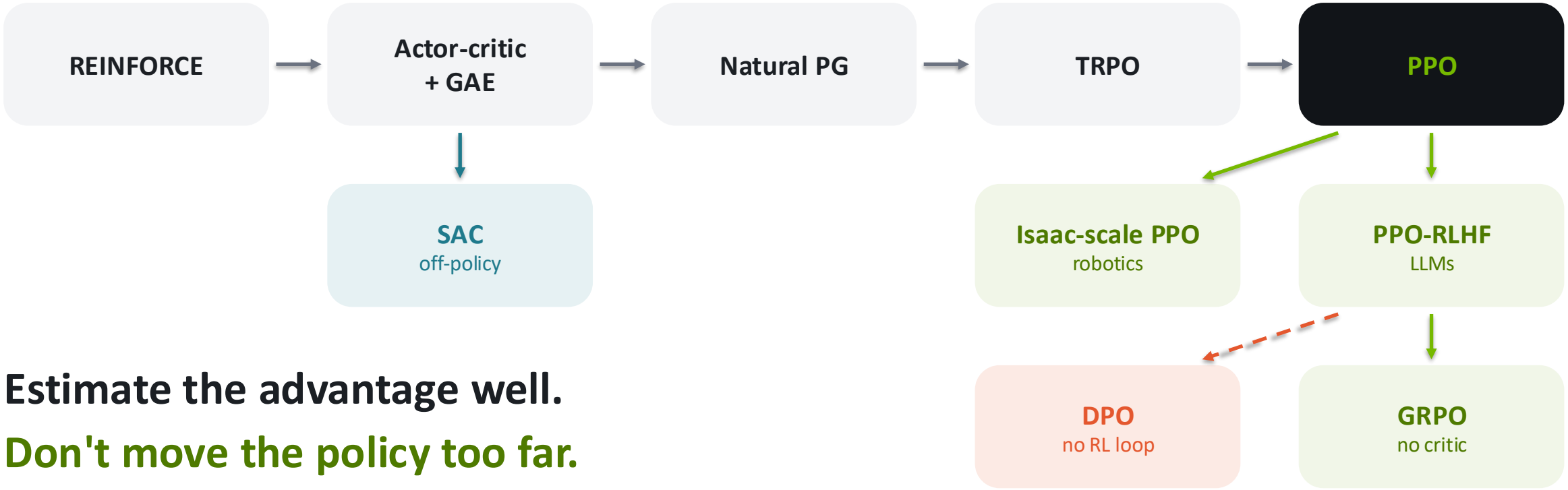
No sampling
No critic
No RL loop

But: offline

fixed preference data → no exploration

Rafailov et al., 2023

One idea, many algorithms



Estimate the advantage well.

Don't move the policy too far.

UP NEXT · 16:10

Your turn: implement PPO

```
def compute_ppo_loss(
        old_log_probs, new_log_probs,
        advantages, clip_eps=0.2):

    # TODO 5: clipped surrogate
    #         3-4 lines
    ...
```

1. Read main.py end-to-end
2. Fill the 5 TODOs, run tester.py
3. REINFORCE vs. PPO on CartPole & LunarLander

Extensions: MountainCar — why does it fail? · GAE λ sweep

github.com/ido90/toy-ppo